

Deep Learning

Lecture 5: Convolutional Networks

Room: **BIN-1-B.01**

Instructor: Manuel Günther

Email: guenther@ifi.uzh.ch

Office: AND 2.54

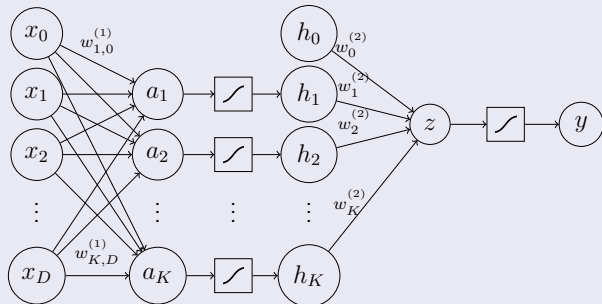
Friday, March 20, 2026, 8:15 – 10:00

Outline

- 1 Review of last Lecture
- 2 Automatic Differentiation
- 3 1D Convolutional Networks
- 4 Multi-Dimensional Convolution
- 5 Convolutional Networks

Review of last Lecture

Binary Classification Network



Learning Binary Classification

- Introduce **logit** $z = \vec{w}^{(2)\top} \vec{h}$
- Compute output $y = \sigma(z)$
- Binary cross-entropy loss
- Its derivative

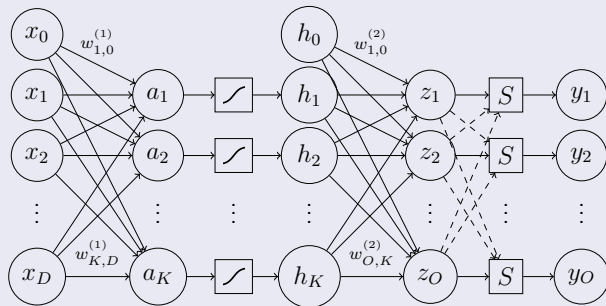
$$\frac{\partial \mathcal{J}^{\text{BCE}}}{\partial z} = y - t$$

Binary Cross-Entropy Loss

$$\mathcal{J}^{\text{BCE}} = - \sum_{n=1}^N \left[t^{[n]} \log y^{[n]} + (1 - t^{[n]}) \log(1 - y^{[n]}) \right]$$

Review of last Lecture

Categorical Classification Network



Categorical Classification

- Logit vector $\vec{z} = \mathbf{W}^{(2)\top} \vec{h}$
- SoftMax $\vec{y} = S(\vec{z})$
- Categorical cross-entropy
- Its derivative

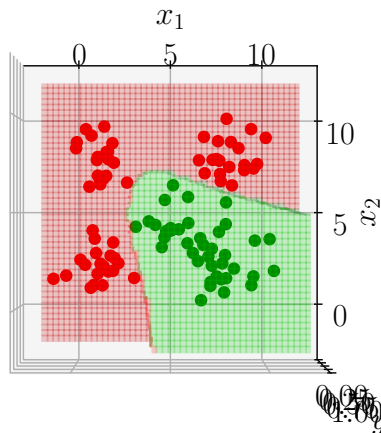
$$\frac{\partial \mathcal{J}^{\text{CCE}}}{\partial z_o} = y_o - t_o$$

Categorical Cross-Entropy Loss of SoftMax Activation

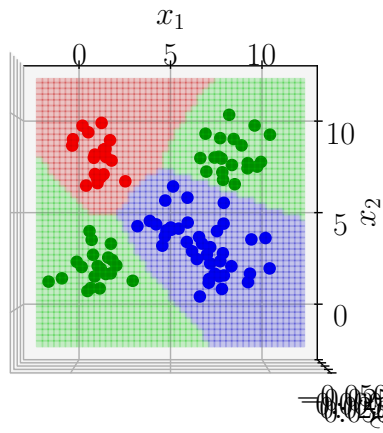
$$\mathcal{J}^{\text{CCE}} = - \sum_{n=1}^N \sum_{o=1}^O t_o^{[n]} \log y_o^{[n]} = - \sum_{n=1}^N \left[z_{\tau^{[n]}}^{[n]} - \log \sum_{o=1}^O e^{z_o^{[n]}} \right]$$

Review of last Lecture

Binary Classification



Categorical Classification



Review of Two-Layer Network Backpropagation

Required Derivatives

- Looking at second layer output
- $\frac{\partial z_o^{[n]}}{\partial w_{o,k}^{(2)}}$ for second layer gradient
- $\frac{\partial z_o^{[n]}}{\partial h_k^{[n]}}$ for first layer gradient
 - Derivatives for all function inputs
- ⇒ General rule for backpropagation
- Exception: first layer and loss
 - $\frac{\partial a_k^{[n]}}{\partial x_d}$ and $\frac{\partial \mathcal{J}^{[n]}}{\partial t_o}$ not required

Second Layer Output

$$\vec{z}^{[n]} = \mathbf{W}^{(2)} \vec{h}^{[n]}$$

Second Layer Gradient of $\mathcal{J}^{\text{CCE}}$

$$\frac{\partial \mathcal{J}^{[n]}}{\partial w_{o,k}^{(2)}} = \frac{\partial \mathcal{J}^{[n]}}{\partial z_o^{[n]}} \frac{\partial z_o^{[n]}}{\partial w_{o,k}^{(2)}}$$

First Layer Gradient of $\mathcal{J}^{\text{CCE}}$

$$\frac{\partial \mathcal{J}^{[n]}}{\partial w_{k,d}^{(1)}} = \sum_{o=1}^O \frac{\partial \mathcal{J}^{[n]}}{\partial z_o^{[n]}} \frac{\partial z_o^{[n]}}{\partial h_k^{[n]}} \frac{\partial h_k^{[n]}}{\partial a_k^{[n]}} \frac{\partial a_k^{[n]}}{\partial w_{k,d}^{(1)}}$$

How to automate this?

A Simplified Example

A Very Simple Network

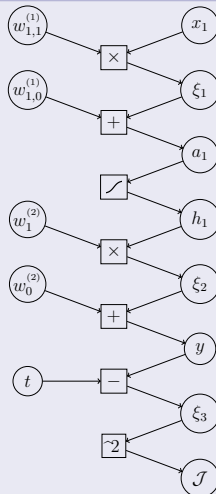
- Four different weights
- One input x_1 and bias
- Logistic activation σ
- One hidden neuron and bias
- One output y , target t , loss $\mathcal{J}$
- Slack variables

→ $\xi_1, a_1, h_1, \xi_2, y$ and ξ_3

Unfolded Loss Equation

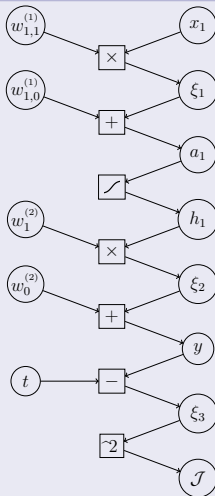
$$\mathcal{J} = \left(w_0^{(2)} + w_1^{(2)} \sigma(w_{1,0}^{(1)} + w_{1,1}^{(1)} x_1) - t \right)^2$$

Computation Graph



A Simplified Example

Computation Graph



Building the Computation Graph

- Ways of graph buildup:
 - Static, during construction: Tensorflow
 - Dynamic, while execution: PyTorch
- Operation execution stores operands
- Operation computes derivatives
 - Derive all outputs to all inputs
 - Some inputs do not require gradients
- Derivative chain is multiplied

Further Reading

Goodfellow Section 6.5

A Simplified Example

Computing the Derivative

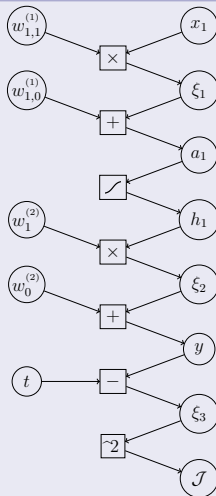
- Traversal of computation graph
 - Start at the end ($\mathcal{J}$)
 - Arrive at each weight w leaf
 - Ignore non-weight leaves
- Multiply partial derivatives

Unfolded Loss Equation

$$\mathcal{J} = \left(w_0^{(2)} + w_1^{(2)} \sigma(w_{1,0}^{(1)} + w_{1,1}^{(1)} x_1) - t \right)^2$$

Derivatives

Computation Graph



A Simplified Example

Unfolded Loss Equation

$$\mathcal{J} = \left(w_0^{(2)} + w_1^{(2)} \sigma(w_{1,0}^{(1)} + w_{1,1}^{(1)} x_1) - t \right)^2$$

Derivatives

$$\frac{\partial \mathcal{J}}{\partial \xi_3} = \frac{\partial (\xi_3)^2}{\partial \xi_3} = 2\xi_3$$

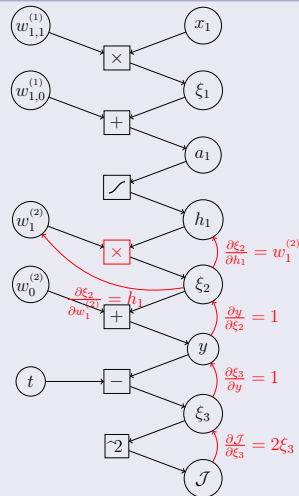
$$\frac{\partial \xi_3}{\partial y} = \frac{\partial (y - t)}{\partial y} = 1$$

$$\frac{\partial \xi_3}{\partial t} = -1$$

$$\frac{\partial y}{\partial w_0^{(2)}} = \frac{\partial (w_0^{(2)} + \xi_2)}{\partial w_0^{(2)}} = 1 \quad \frac{\partial y}{\partial \xi_2} = \frac{\partial (w_0^{(2)} + \xi_2)}{\partial \xi_2} = 1$$

$$\frac{\partial \xi_2}{\partial w_1^{(2)}} = \frac{\partial w_1^{(2)} h_1}{\partial w_1^{(2)}} = h_1 \quad \frac{\partial \xi_2}{\partial h_1} = \frac{\partial w_1^{(2)} h_1}{\partial h_1} = w_1^{(2)}$$

Computation Graph



A Simplified Example

Unfolded Loss Equation

$$\mathcal{J} = \left(w_0^{(2)} + w_1^{(2)} \sigma(w_{1,0}^{(1)} + w_{1,1}^{(1)} x_1) - t \right)^2$$

Derivatives

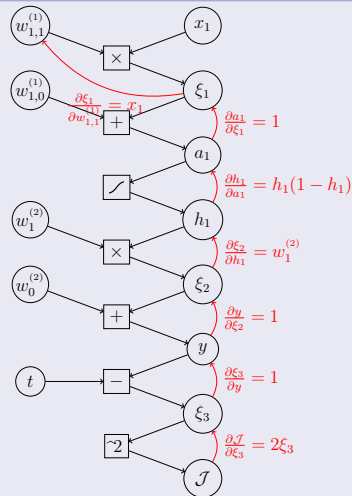
$$\frac{\partial a_1}{\partial w_{1,0}^{(1)}} = \frac{\partial(w_{1,0}^{(1)} + \xi_1)}{\partial w_{1,0}^{(1)}} = 1 \quad \frac{\partial a_1}{\partial w_{1,0}^{(1)}} = \frac{\partial(w_{1,0}^{(1)} + \xi_1)}{\partial \xi_1} = 1$$

$$\frac{\partial \xi_1}{\partial w_{1,1}^{(1)}} = \frac{\partial w_{1,1}^{(1)} x_1}{\partial w_{1,1}^{(1)}} = x_1 \quad \frac{\partial \xi_1}{\partial x_1}$$

$$\frac{\partial \mathcal{J}}{\partial w_1^{(2)}} = 2\xi_3 1 h_1$$

$$\frac{\partial \mathcal{J}}{\partial w_{1,1}^{(1)}} = 2\xi_3 1 w_1^{(2)} h_1 (1 - h_1) 1 x_1$$

Computation Graph



Vectorizing Automatic Differentiation

Vectorization

- Above example: single variables
 - Difficult to track
 - Expensive to evaluate
- Define multi-input and multi-output operations
- Compute the Jacobian matrices

The Jacobian Matrix

- Assuming $f : \mathbb{R}^M \rightarrow \mathbb{R}^N$
- Computing $\vec{y} = f(\vec{x})$
- Matrix of partial derivatives:

$$J_f = \frac{\partial \vec{y}}{\partial \vec{x}} = \left(\frac{\partial y_i}{\partial x_j} \right)_{\substack{i=1,\dots,N \\ j=1,\dots,M}}$$

- Shape: $J_f \in \mathbb{R}^{N \times M}$

Baydin, Pearlmutter, Radul, Siskind: “Automatic Differentiation in Machine Learning: a Survey” JMLR 2018

Vectorizing Automatic Differentiation

Example: Second Layer

- Matrix-vector multiplication:

$$\vec{z} = \mathbf{W}^{(2)} \vec{h} \quad z_o = \sum_{k=0}^K w_{o,k}^{(2)} h_k$$

- Two** Jacobian matrices:

$$\frac{\partial \vec{z}}{\partial \vec{h}} \quad \text{and} \quad \frac{\partial \vec{z}}{\partial \mathbf{W}^{(2)}}$$

→ Elements of $\mathbf{W}^{(2)}$ interpreted as vector of weights

First Jacobian Matrix

$$\frac{\partial \vec{z}}{\partial \vec{h}} = \begin{pmatrix} \frac{\partial z_1}{\partial h_0} & \frac{\partial z_1}{\partial h_1} & \dots & \frac{\partial z_1}{\partial h_K} \\ \frac{\partial z_2}{\partial h_0} & \frac{\partial z_2}{\partial h_1} & \dots & \frac{\partial z_2}{\partial h_K} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial z_O}{\partial h_0} & \frac{\partial z_O}{\partial h_1} & \dots & \frac{\partial z_O}{\partial h_K} \end{pmatrix}$$

$$= \mathbf{W}^{(2)} \in \mathbb{R}^{O \times (K+1)}$$

Vectorizing Automatic Differentiation

Second Jacobian Matrix

$$\frac{\partial \vec{z}}{\partial \mathbf{W}^{(2)}} = \begin{pmatrix} \frac{\partial z_1}{\partial w_{1,0}^{(2)}} & \frac{\partial z_1}{\partial w_{1,1}^{(2)}} & \cdots & \frac{\partial z_1}{\partial w_{1,K}^{(2)}} & \frac{\partial z_1}{\partial w_{2,0}^{(2)}} & \cdots & \frac{\partial z_1}{\partial w_{O,K}^{(2)}} \\ \frac{\partial z_2}{\partial w_{1,0}^{(2)}} & \frac{\partial z_2}{\partial w_{1,1}^{(2)}} & \cdots & \frac{\partial z_2}{\partial w_{1,K}^{(2)}} & \frac{\partial z_2}{\partial w_{2,0}^{(2)}} & \cdots & \frac{\partial z_2}{\partial w_{O,K}^{(2)}} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ \frac{\partial z_O}{\partial w_{1,0}^{(2)}} & \frac{\partial z_O}{\partial w_{1,1}^{(2)}} & \cdots & \frac{\partial z_O}{\partial w_{1,K}^{(2)}} & \frac{\partial z_O}{\partial w_{2,0}^{(2)}} & \cdots & \frac{\partial z_O}{\partial w_{O,K}^{(2)}} \end{pmatrix} \in \mathbb{R}^{O \times O(K+1)}$$

$$= \begin{pmatrix} h_0 & h_1 & \cdots & h_K & 0 & \cdots & 0 & \cdots & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 & h_0 & \cdots & h_K & \cdots & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \cdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & 0 & \cdots & 0 & \cdots & h_0 & \cdots & h_K \end{pmatrix}$$

Vectorizing Automatic Differentiation

Putting Things Together

- What we have:
 - Jacobian matrices for each operation
 - Computation graph of operations
- What we need:
 - Computing gradients w.r.t. weights

Second Layer Gradient

$$\frac{\partial \mathcal{J}}{\partial \mathbf{W}^{(2)}} = \frac{1}{N} \sum_{n=1}^N \frac{\partial \mathcal{J}}{\partial \mathcal{J}^{[n]}} \frac{\partial \mathcal{J}^{[n]}}{\partial \vec{z}^{[n]}} \frac{\partial \vec{z}^{[n]}}{\partial \mathbf{W}^{(2)}}$$

First Layer Gradient

$$\frac{\partial \mathcal{J}}{\partial \mathbf{W}^{(1)}} = \frac{1}{N} \sum_{n=1}^N \frac{\partial \mathcal{J}}{\partial \mathcal{J}^{[n]}} \frac{\partial \mathcal{J}^{[n]}}{\partial \vec{z}^{[n]}} \frac{\partial \vec{z}^{[n]}}{\partial \vec{h}^{[n]}} \frac{\partial \vec{h}^{[n]}}{\partial \vec{a}^{[n]}} \frac{\partial \vec{a}^{[n]}}{\partial \mathbf{W}^{(1)}}$$

Vectorizing Automatic Differentiation

Further Vectorizing

- Sum over samples can be turned into matrix representation

$$\frac{\partial \mathcal{J}}{\partial \mathbf{W}^{(2)}} = \frac{\partial \mathcal{J}}{\partial \mathbf{Z}} \frac{\partial \mathbf{Z}}{\partial \mathbf{W}^{(2)}} \quad \frac{\partial \mathcal{J}}{\partial \mathbf{W}^{(1)}} = \frac{\partial \mathcal{J}}{\partial \mathbf{Z}} \frac{\partial \mathbf{Z}}{\partial \mathbf{H}} \frac{\partial \mathbf{H}}{\partial \mathbf{A}} \frac{\partial \mathbf{A}}{\partial \mathbf{W}^{(1)}}$$

- Theoretical sizes of (extremely sparse) Jacobian matrices

$$\frac{\partial \mathcal{J}}{\partial \mathbf{Z}} \in \mathbb{R}^{1 \times ON} \quad \frac{\partial \mathbf{Z}}{\partial \mathbf{W}^{(2)}} \in \mathbb{R}^{ON \times O(K+1)}$$

$$\frac{\partial \mathbf{Z}}{\partial \mathbf{H}} \in \mathbb{R}^{ON \times (K+1)N} \quad \frac{\partial \mathbf{H}}{\partial \mathbf{A}} \in \mathbb{R}^{(K+1)N \times (K+1)N} \quad \frac{\partial \mathbf{A}}{\partial \mathbf{W}^{(1)}} \in \mathbb{R}^{(K+1)N \times (D+1)(K+1)}$$

Automatic Differentiation in Practice

How to Handle Large Derivative Chains

- Compute matrix multiplications only once each
- Store **combined** partial derivatives

Both Layers Derivative Chain

$$\frac{\partial \mathcal{J}}{\partial \mathbf{W}^{(2)}} = \frac{\partial \mathcal{J}}{\partial \mathbf{Z}} \frac{\partial \mathbf{Z}}{\partial \mathbf{W}^{(2)}}$$

$$\frac{\partial \mathcal{J}}{\partial \mathbf{W}^{(2)}} = \frac{\partial \mathcal{J}}{\partial \mathbf{Z}} \frac{\partial \mathbf{Z}}{\partial \mathbf{W}^{(2)}}$$

$$\dim \left(\frac{\partial \mathcal{J}}{\partial \dots} \right) = \dim(\dots)$$

$$\frac{\partial \mathcal{J}}{\partial \mathbf{Z}}$$

$$\frac{\partial \mathcal{J}}{\partial \mathbf{W}^{(1)}} = \frac{\partial \mathcal{J}}{\partial \mathbf{Z}} \frac{\partial \mathbf{Z}}{\partial \mathbf{H}} \frac{\partial \mathbf{H}}{\partial \mathbf{A}} \frac{\partial \mathbf{A}}{\partial \mathbf{W}^{(1)}}$$

$$\frac{\partial \mathcal{J}}{\partial \mathbf{H}} = \frac{\partial \mathcal{J}}{\partial \mathbf{Z}} \frac{\partial \mathbf{Z}}{\partial \mathbf{H}}$$

$$\frac{\partial \mathcal{J}}{\partial \mathbf{A}} = \frac{\partial \mathcal{J}}{\partial \mathbf{H}} \frac{\partial \mathbf{H}}{\partial \mathbf{A}}$$

$$\frac{\partial \mathcal{J}}{\partial \mathbf{W}^{(1)}} = \frac{\partial \mathcal{J}}{\partial \mathbf{A}} \frac{\partial \mathbf{A}}{\partial \mathbf{W}^{(1)}}$$

1D Convolutional Kernels

Fully-Connected Layer

- Assumes independence of input dimensions
 - Not applicable to images
- Many parameters: $K \times D$
 - Consumes lots of memory
- Fixed input and output size

Convolutional Operations

- Models dependencies between input dimensions
 - Can be applied to images
- Fewer parameters: U
 - Less prone to overfitting
- Variable input dimensions

1D Convolutional Kernels

Discrete Convolution (1D)

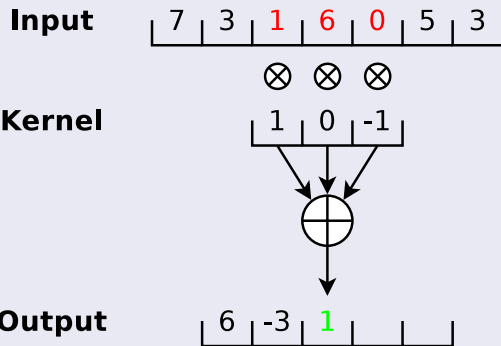
- Input: $\vec{x} \in \mathbb{R}^D$ (no bias x_0)
- Kernel: $\vec{w} \in \mathbb{R}^U$ (often U odd)
- Output: $\vec{a} \in \mathbb{R}^K$ ($K = D - U + 1$)
- Convolution: $\vec{a} = \vec{x} * \vec{w}$
- Single element (via definition):

$$a_k = (\vec{x} * \vec{w})_k = \sum_{u=0}^{U-1} x_{k+u} \cdot w_{U-u}$$

- Single element (as applied):

$$a_k = \sum_{u=1}^U x_{k+u-1} \cdot w_{\textcolor{red}{u}}$$

Example 1D Convolution



Further Reading

Skansi Ch. 6; Goodfellow Ch. 9

1D Convolutional Kernels

Variation 1: Zero Padding

- “Add” 0 left and right to $\vec{x}$

→ Implemented differently:

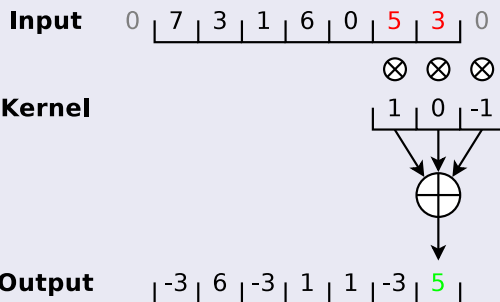
$$a_k = \sum_{u=1}^U \mathbb{I}(1 < k+u \leq D) \cdot x_{k+u-1} \cdot w_u$$

- Keep dimensions: $K = D$
→ But introduces artificial borders

Variation 2: Stride

- Reduce dimensions: $K = \lceil \frac{D}{2} \rceil$
- Skip onset positions
→ Might skip important values
→ Better alternatives ahead

1D Convolution with Padding



1D Convolutional Kernels

Variation 1: Zero Padding

- “Add” 0 left and right to $\vec{x}$

→ Implemented differently:

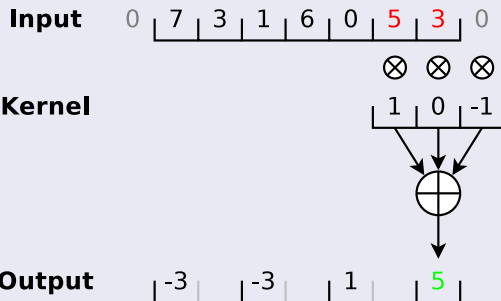
$$a_k = \sum_{u=1}^U \mathbb{I}(1 < k+u \leq D) \cdot x_{k+u-1} \cdot w_u$$

- Keep dimensions: $K = D$
→ But introduces artificial borders

Variation 2: Stride

- Reduce dimensions: $K = \left\lceil \frac{D}{2} \right\rceil$
- Skip onset positions
→ Might skip important values
→ Better alternatives ahead

1D Convolution with Stride



1D Convolutional Networks

Convolutional Kernels in Networks

- Replace fully-connected layers
- Usually applied in first layer(s)
- Act as receptive fields
- Reduce number of parameters
- Added bias term:

$$a_k = w_0 + \sum_{u=1}^U x_{k+u-1} \cdot w_u$$

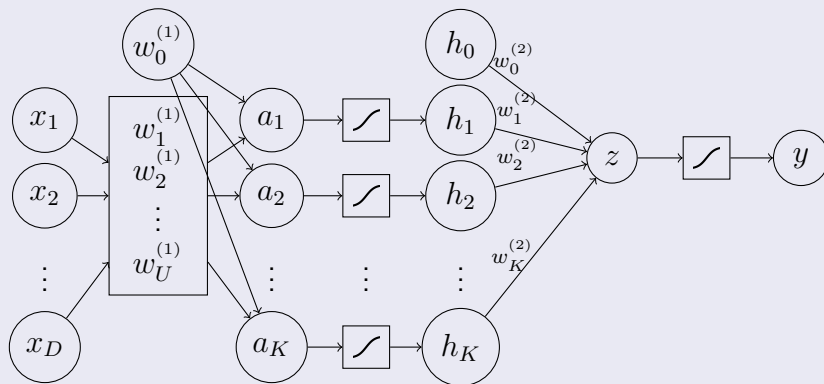
- Activation function $\vec{h} = g^{(1)}(\vec{a})$

Independent of Output and Loss

- Mix-and-match as required
 - Convolution $\vec{w}^{(1)}$
+ multi-output $\mathbf{W}^{(2)}$
+ regression $\mathcal{J}^{L_2}$
 - Fully-connected $\mathbf{W}^{(1)}$
+ single-output $\vec{w}^{(2)}$
+ binary classification $\mathcal{J}^{\text{BCE}}$
 - Convolution $\vec{w}^{(1)}$
+ multi-output $\mathbf{W}^{(2)}$
+ categorical classification $\mathcal{J}^{\text{CCE}}$
 - ...

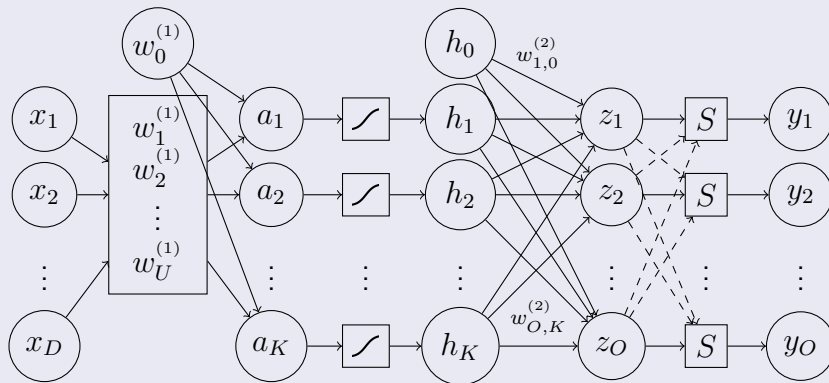
1D Convolutional Networks

Convolutional Binary Classification Network



1D Convolutional Networks

Convolutional Categorical Classification Network



Derivative of 1D Convolution

Generic Representation of Two-Layer Network and Loss

$$\vec{y} = g^{(2)} \left(\underset{\vec{z}}{\wedge} f^{(2)} \left(\underset{\vec{h}}{\wedge} g^{(1)} \left(\underset{\vec{a}}{\wedge} f^{(1)}(\vec{x}) \right) \right) \right) \quad \mathcal{J} = \sum_{n=1}^N \text{Cmp}(\vec{y}^{[n]}, \vec{t}^{[n]})$$

Recall: Derivative Chain of Fully-Connected Network

$$\frac{\partial \mathcal{J}}{\partial w_{o,k}^{(2)}} = \sum_{n=1}^N \frac{\partial \mathcal{J}^{[n]}}{\partial z_o^{[n]}} \frac{\partial z_o^{[n]}}{\partial w_{o,k}^{(2)}} \quad \checkmark \quad \frac{\partial \mathcal{J}}{\partial w_{k,d}^{(1)}} = \sum_{n=1}^N \sum_{o=1}^O \frac{\partial \mathcal{J}^{[n]}}{\partial z_o^{[n]}} \frac{\partial z_o^{[n]}}{\partial h_k^{[n]}} \frac{\partial h_k^{[n]}}{\partial a_k^{[n]}} \frac{\partial a_k^{[n]}}{\partial w_{k,d}^{(1)}}$$

1D Convolution

$$a_k = w_0 + \sum_{u=1}^U x_{k+u-1} \cdot w_u$$

Derivative Chain of 1D Convolutional Network

$$\frac{\partial \mathcal{J}}{\partial w_u^{(1)}} = \sum_{n=1}^N \sum_{o=1}^O \frac{\partial \mathcal{J}^{[n]}}{\partial z_o^{[n]}} \sum_{k=1}^K \frac{\partial z_o^{[n]}}{\partial h_k^{[n]}} \frac{\partial h_k^{[n]}}{\partial a_k^{[n]}} \frac{\partial a_k^{[n]}}{\partial w_u^{(1)}}$$

Derivative of 1D Convolution

1D Convolution

$$a_k = w_0 + \sum_{u=1}^U x_{k+u-1} \cdot w_u$$

Derivative

$$\frac{\partial a_k^{[n]}}{\partial w_u^{(1)}} = \frac{\partial \sum_{u'=1}^U x_{k+u'-1} \cdot w_{u'}^{(1)}}{\partial w_u^{(1)}} = x_{k+u-1}$$

Derivative Chain of 1D Convolutional Network

$$\begin{aligned} \frac{\partial \mathcal{J}}{\partial w_u^{(1)}} &= \sum_{n=1}^N \sum_{o=1}^O \frac{\partial \mathcal{J}^{[n]}}{\partial z_o^{[n]}} \sum_{k=1}^K \frac{\partial z_o^{[n]}}{\partial h_k^{[n]}} \frac{\partial h_k^{[n]}}{\partial a_k^{[n]}} \frac{\partial a_k^{[n]}}{\partial w_u^{(1)}} \\ &= \sum_{n=1}^N \sum_{o=1}^O 2(y_o^{[n]} - t_o^{[n]}) \sum_{k=1}^K w_{o,k}^{(2)} h_k^{[n]} (1 - h_k^{[n]}) x_{k+u-1}^{[n]} \end{aligned}$$

Further Reading

Not provided in any textbook

Interpretation as Matrix Multiplication

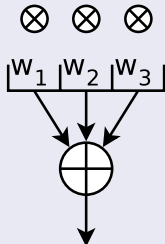
Example: $K = 3$, $D = 7$, $\text{pad}=1$

Input 0

x_1	x_2	x_3	x_4	x_5	x_6	x_7
-------	-------	-------	-------	-------	-------	-------

 0

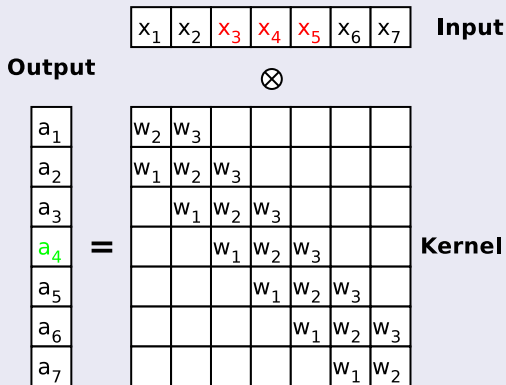
Kernel



Output

a_1	a_2	a_3	a_4	a_5	a_6	a_7
-------	-------	-------	-------	-------	-------	-------

Convolution as Multiplication



We know how to compute Jacobians for matrix multiplication
 $\Rightarrow$ Collect derivatives for all occurrences of variable in Jacobian

Pooling

Replacement for Stride

- Applied after convolution
- Incorporate several outputs
- Define region R to combine
→ Overlap possible, but uncommon

Variations of Pooling

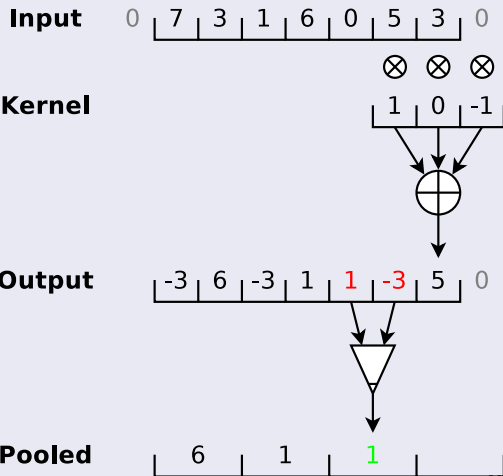
- Average pooling:

$$a_k = \frac{1}{R} \sum_{j=1}^R \hat{a}_{R(k-1)+j}$$

- Maximum pooling:

$$a_k = \max_{j \in \{1, \dots, R\}} \hat{a}_{R(k-1)+j}$$

1D Convolution with Pooling



2D Convolution

Image Processing

- Images are 2D: $\mathbf{X} \in \mathbb{R}^{D \times E}$
- 2D convolution kernels: $\mathbf{W} \in \mathbb{R}^{U \times V}$
- 2D convolution: $\mathbf{A} \in \mathbb{R}^{K \times M} = \mathbf{X} * \mathbf{W}$

$$a_{k,m} = \sum_{u=1}^U \sum_{v=1}^V x_{k+u-1, m+v-1} \cdot w_{u,v}$$

- Variants apply:
 - Zero padding
 - Strides / Pooling
- Independent of order
 - Parallelizable for all output cells $a_{k,m}$

2D Convolution

2D Convolution Example with Zero Padding

Input

0	0	0	0	0	0	0
0	4	2	6	1	0	0
0	1	4	3	7	1	0
0	8	0	1	4	9	0
0	7	1	2	5	8	0
0	0	0	0	0	0	0

Kernel

5	1	-3
4	-2	0
2	-1	6

Output

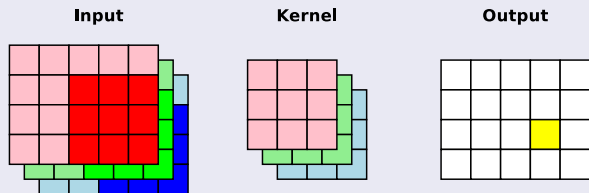
15	28	43	27	17
-12	22	46	81	30
-28	57	30	62	36
-6	63	-11	-20	33

2D Convolution with Channels

Working with Color Images

- Color images are 3D *tensors*: $\mathcal{X} \in \mathbb{R}^{C \times D \times E}$
 - For RGB images: $C = 3$ channels for Red, Green, Blue
- 3D convolution kernels: $\mathcal{W} \in \mathbb{R}^{C \times U \times V}$
 - Note: C is the same as for the input $\mathcal{X}$
- 3D input to 2D output: $\mathbf{A} \in \mathbb{R}^{K \times M} = \mathcal{X} * \mathcal{W}$

Channeled Convolution Example



Convolution Variants

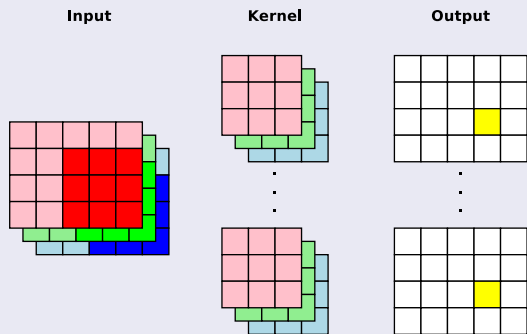
- Zero padding
- Strides / Pooling

2D Convolutions with Channels

Multiple Convolutions

- Single filter insufficient
- In human visual system:
 - Several scale and orientation-selective filters
- In convolutional layer:
 - Several learned filters
 $\mathcal{W} = \{\mathcal{W}_1, \dots, \mathcal{W}_Q\}$
- 3D input: $\mathcal{X} \in \mathbb{R}^{C \times D \times E}$
- 4D weights: $\mathcal{W} \in \mathbb{R}^{Q \times C \times U \times V}$
- 3D output: $\mathcal{A} \in \mathbb{R}^{Q \times K \times M}$
 - Also known as *feature map*

Channeled Convolution Example



Channeled Convolution Result

$$a_{q,k,m} = w_q + \sum_{c=1}^C \sum_{u=1}^U \sum_{v=1}^V x_{c,k+u-1,m+v-1} \cdot w_{q,c,u,v}$$

2D Convolutions with Channels

Derivative of Multiple Channeled 2D Convolution

$$\frac{\partial a_{q,k,m}}{\partial w_{q,c,u,v}} = \frac{\partial \sum_{c'=1}^C \sum_{u'=1}^U \sum_{v'=1}^V x_{c',k+u'-1,m+v'-1} \cdot w_{q,c',u',v'}}{\partial w_{q,c,u,v}} = x_{c,k+u-1,m+v-1}$$

Derivative Chain

- Several sums will stay inside the derivative chain
- Fast implementation with sparse matrix multiplication

Stacked Convolutions

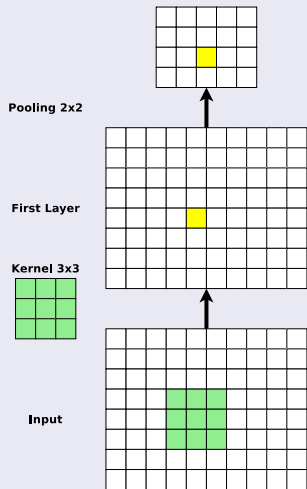
Receptive Fields

- Kernels smaller than images:
 $U \ll D \wedge V \ll E$
- Kernels “see” few pixels
→ Impossible to learn large structures

Solutions

- Larger kernels
→ Increased number of weights, quadratic in size
- Stacked convolutions + pooling
→ #Weights linear in #layers
→ Dramatically increased field of view

Convolution+Pooling



Stacked Convolutions

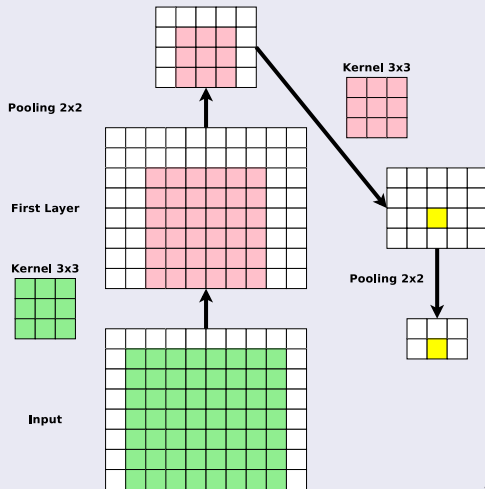
Receptive Fields

- Kernels smaller than images:
 $U \ll D \wedge V \ll E$
- Kernels “see” few pixels
→ Impossible to learn large structures

Solutions

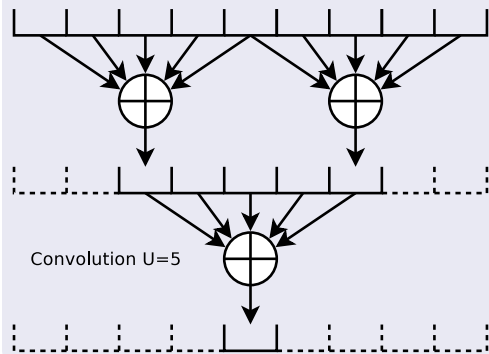
- Larger kernels
→ Increased number of weights, quadratic in size
- Stacked convolutions + pooling
→ #Weights linear in #layers
→ Dramatically increased field of view

Stacked Convolution+Pooling

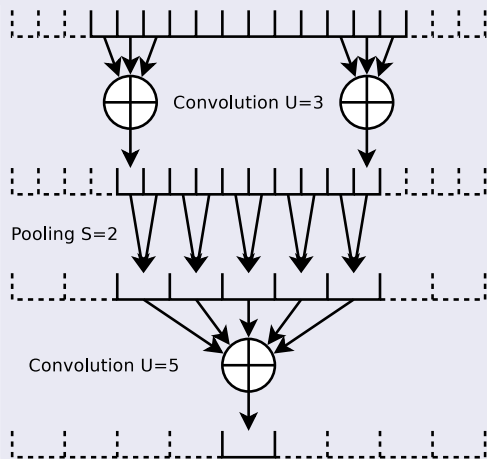


Receptive Field Estimation

Stacked Convolution Layers



Stacked Conv. + Pooling



Stacked Convolutions

Stacked Convolutional Layers

- Convolutional layer as first layer

$$a_{q,k,m}^{(1)} = w_q^{(1)} + \sum_{c=1}^C \sum_{u=1}^{U^{(1)}} \sum_{v=1}^{V^{(1)}} x_{c,k+u-1,m+v-1} \cdot w_{q,c,u,v}^{(1)}$$

- Apply activation function $\mathcal{H}^{(1)} = g^{(1)}(\mathcal{A}^{(1)})$
- Stacked convolutional layer as second layer

$$a_{q,k,m}^{(2)} = w_q^{(2)} + \sum_{c=1}^{Q^{(1)}} \sum_{u=1}^{U^{(2)}} \sum_{v=1}^{V^{(2)}} h_{c,k+u-1,m+v-1}^{(1)} \cdot w_{q,c,u,v}^{(2)}$$

- Derivatives $\Rightarrow$ automatic differentiation

Required Derivatives

$$\frac{\partial \mathcal{A}^{(1)}}{\partial \mathcal{W}^{(1)}} \equiv \frac{\partial a_{q,k,m}^{(1)}}{\partial w_{q,c,u,v}^{(1)}}$$

$$\frac{\partial \mathcal{A}^{(2)}}{\partial \mathcal{W}^{(2)}} \equiv \frac{\partial a_{q,k,m}^{(2)}}{\partial w_{q,c,u,v}^{(2)}}$$

$$\frac{\partial \mathcal{A}^{(2)}}{\partial \mathcal{H}^{(1)}} \equiv \frac{\partial a_{q,k,m}^{(2)}}{\partial h_{q',k',m'}^{(1)}}$$

Convolutional Networks

Connecting Convolution and Fully-Connected layers

- Output of convolution multi-dimensional (2D or 3D)
- Input to fully-connected layer only vectorial
 - ⇒ Turn $\mathcal{A} \in \mathbb{R}^{Q \times K \times M}$ to $\vec{a} \in \mathbb{R}^{Q \cdot K \cdot M}$ by **flattening**
 - ⇒ Turn $\mathcal{A} \in \mathbb{R}^{Q \times K \times M}$ to $\vec{a} \in \mathbb{R}^Q$ by **global pooling**

Indexing

0	1	2	3	4				
5	6	7	8	9	24			
10	11	12	13	14	29	44		
15	16	17	18	19	34	49	64	
	35	36	37	38	39	54	69	
		55	56	57	58	59	74	
			75	76	77	78	79	

How Flattening works in Practice

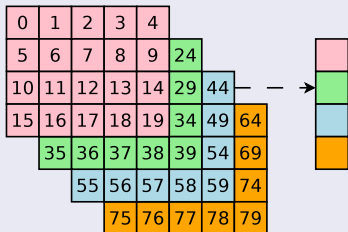
- Memory (RAM) is one-dimensional
- Compute 1D index from 3D index
 - ⇒ Flattening = reinterpretation of data
 - No data copying required

Convolutional Networks

Connecting Convolution and Fully-Connected layers

- Output of convolution multi-dimensional (2D or 3D)
- Input to fully-connected layer only vectorial
 - ⇒ Turn $\mathcal{A} \in \mathbb{R}^{Q \times K \times M}$ to $\vec{a} \in \mathbb{R}^{Q \cdot K \cdot M}$ by **flattening**
 - ⇒ Turn $\mathcal{A} \in \mathbb{R}^{Q \times K \times M}$ to $\vec{a} \in \mathbb{R}^Q$ by **global pooling**

Global Pooling



Global Pooling: Why and How

- Advantage:
 - Output independent of resolution
 - Objects can be anywhere in input
- Pooling variants:
 - Global Average/Maximum Pooling

Best Practices

How to Design your Network

- Decide kernel size of convolution $U \times V$
 - Typically: $U, V \in \{1, 3, 5, 7, 9, 11\}$; $U = V$, often $U = 3$ or $U = 5$
- Decide number of kernels, typically $Q \in \{2^i \mid 3 \leq i \leq 12\}$
- Number of channels C defined by input
- Often seen: zero-padding of size $\frac{U-1}{2}$
 - Input and output of convolution with same size
- Mostly maximum (rarely average) pooling, often of size $\frac{U+1}{2}$
- Decide for activation function; usually ReLU (see next lecture)
- Decide for the depth of your network
 - More layers preferred over larger or more kernels

LeNet and MNIST

Handwritten Digit Classification

- Large dataset: 70000 samples
→ 60k training, 10k test
- Cropped handwritten digits
→ Writers from USA
- Images of size 28×28
- Almost binary pixels
- 10 different classes:
 $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
→ Almost balanced classes
- Toy dataset for deep learning

Samples



LeNet and MNIST

The Naïve Approach

- Two-Layer network (as before)
- 2D image as 1D vector:
 - $\mathbf{X} \Rightarrow \vec{x}$, $D = 784$ inputs
- Sufficiently many hidden neurons
 - $K \gg 50$
- SoftMax activation
- Categorical cross-entropy loss

Disadvantages

- Pixel correlations ignored
 - Each pixel treated independent of neighbors
 - Network not shift invariant
- Large amounts of weights
 - $\mathbf{W}^{(1)} \in \mathbb{R}^{K+1 \times D+1} \gg 40'000$

The Better Approach

- Convolution kernels

The Rule of Ten for Machine Learning

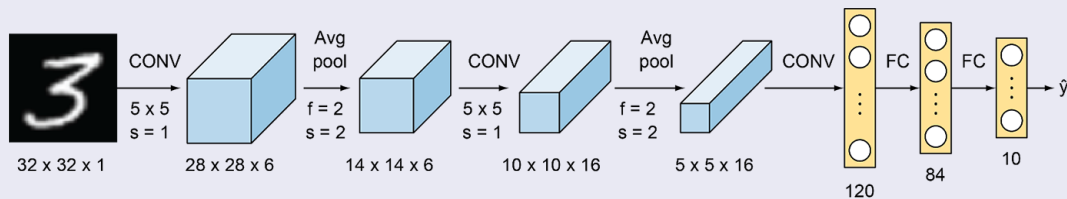
At least 10 times more training data than parameters are required.

LeNet and MNIST

The LeNet Network from 1989

- Does not use zero padding $\Rightarrow$ Convolution result shrinks
- Applied for hand-written digit classification (MNIST)

LeNet Network Topology



LeCun, Boser, Denker, Henderson, Howard, Hubbard, Jackel: "Backpropagation applied to handwritten zip code recognition" Neural Computation, 1989